



山东大学

信息科学与工程学院

2025—2026 学年第一学期

实 验 报 告

课程名称: Java 编程技术

实验名称: 第四次实验

专 业 班 级 通信三班

学 生 学 号

学 生 姓 名

实 验 时 间 2025 年 10 月 25 日

实 验 报 告

【实验目的】

- 1.熟悉 Java 语言的数组与字符串
- 2.熟悉 Java 语言的类与对象

【实验要求】

一、在 Eclipse 平台中实现下列简单的程序

【实验具体内容】

一、数组与字符串

1、随机生成一个 10 个元素组成一维数组，输出数组中的最大值、最小值、所有元素总和，以及最大值和最小值在该数组中的位置，并实现数组的排序。

(1) 实验源程序

```
package cn;

import java.util.Arrays;
import java.util.Random;
public class Arraytest {
    public static void main(String[] args ) {
        Random random =new Random();

        int[] a = new int[10];

        int max =0;
        int min =1000;
        int sum =0;

        for(int i =0; i<10;i++) {
            a[i] = random.nextInt();
            if(a[i] <min) {
                min = a[i];
            }
            if(a[i] >max) {
                max = a[i];
            }
            sum += a[i];
        }

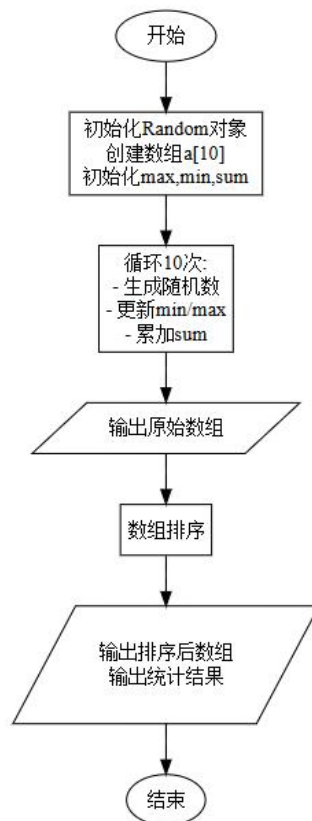
        System.out.println("随机生成的数组"+ Arrays.toString(a)+"\n");
        Arrays.sort(a);
    }
}
```

```

        System.out.println("排序完成后的数组"+ Arrays.toString(a)+"\n");
        System.out.printf("数组和: %d\n",sum);
        System.out.printf("最大值: %d\n",a[9]);
        System.out.printf("最小值: %d\n",a[0]);

    }
}
    
```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：为找到数组中的最大值与最小值，有一种方法是遍历一遍数组，提前设定一个最大值的初始量与最小值的初始量，令其一直比较获得最大值与最小值，但是实际内容中要求我们进行排序，因此我们只要在此基础上按索引输出即可。同时，对于随机数生成，我们需要利用 Random 类的 nextInt 方法来实现。

1) 编写 java 代码并运行

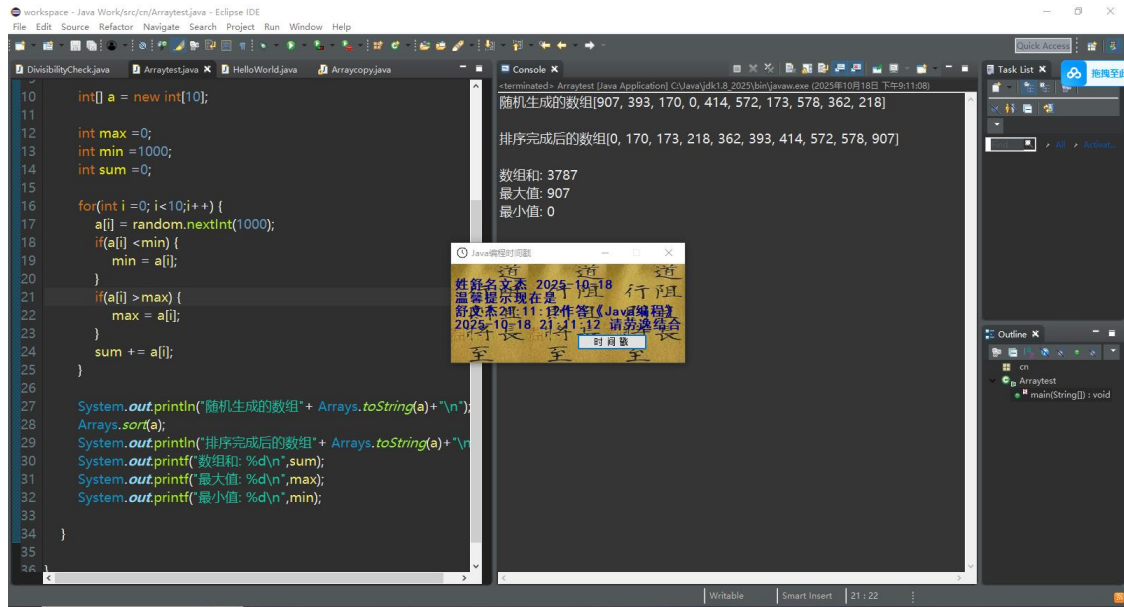


图 1 运行结果截图

此时由于我们使用的思路中的方法一，但我们对最大值与最小值的初始化的值并不恰当，因此，这里我们的 `nextInt` 方法使用的生成随机正整数，为了进行更加满足实验要求，我们使用方案二进行修改。

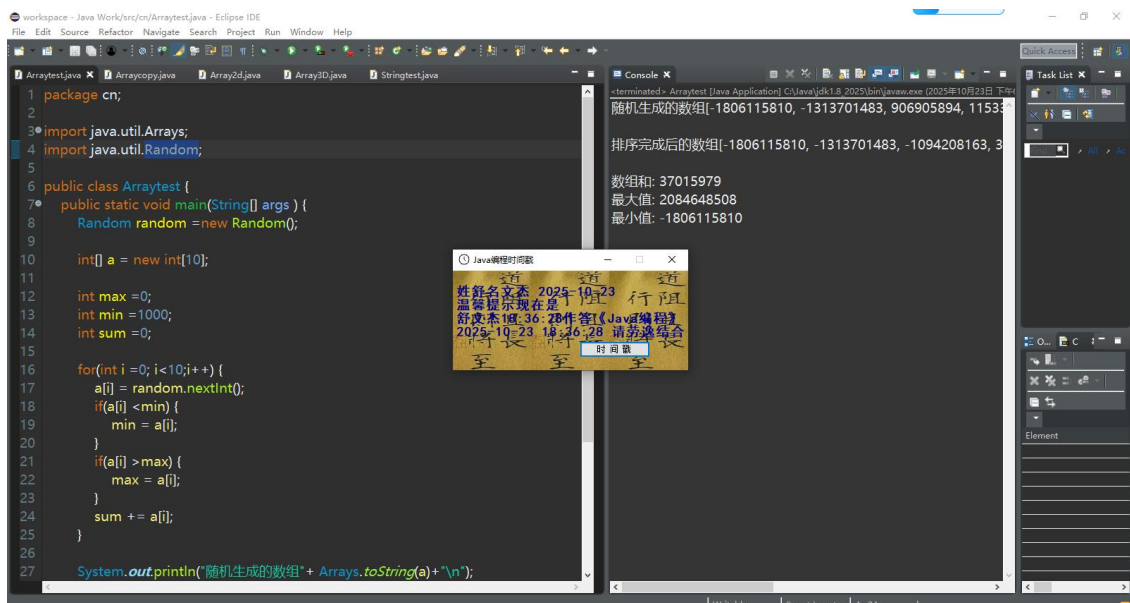


图 2 改进后的实验结果截图

可以发现，我们的运行结果符合预期，我们的实验成功。

2、利用 `arraycopy` 和 `copyof` 方法分别实现两个数组的内容拷贝，并利用 `foreach` 语法规则来遍历两个数组元素的内容。同时学习使用 `foreach` 语法规则来尝试写一个二维数组、三维数组的内容遍历。

(1) 实验源程序

(1) arraycopy 和 copyof 方法实现数组的拷贝

```
package cn;
import java.util.Arrays;

public class Arraycopy {
    public static void main(String[] args) {
        int[] a = {1,2,3,4,5,6};
        int[] b = new int[6];

        System.arraycopy(a, 0, b, 0, 6);

        int[] c = Arrays.copyOf(a, 6);

        System.out.println("原数组: ");
        for (int num : a) {
            System.out.print(num + " ");
        }

        System.out.println("\narraycopy 的复制数组: ");
        for (int num : b) {
            System.out.print(num + " ");
        }

        System.out.println("\nArrays.copy 的复制数组: ");
        for (int num : c) {
            System.out.print(num + " ");
        }

    }
}
```

(2) 使用 foreach 语法实现遍历

2 维矩阵

```
package cn;

public class Array2d {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
    }
}
```

```

        System.out.println("遍历矩阵: ");
        for (int[] row : matrix) {
            for (int column : row) {
                System.out.print(column + " ");
            }
            System.out.println("\n");
        }
    }
}

```

3 维矩阵

```

package cn;

public class Array3D {
    public static void main(String[] args) {
        int[][][] Tensor = {
            {
                {1,2,3},
                {4,5,6},
                {7,8,9}
            }
        };

        System.out.println("遍历张量: ");
        for(int[][] height : Tensor)
            for (int[] row : height) {
                for (int column : row) {
                    System.out.print(column + " ");
                }
                System.out.println("\n");
            }
        System.out.println("\n");
    }
}

```

(2) 代码流程图

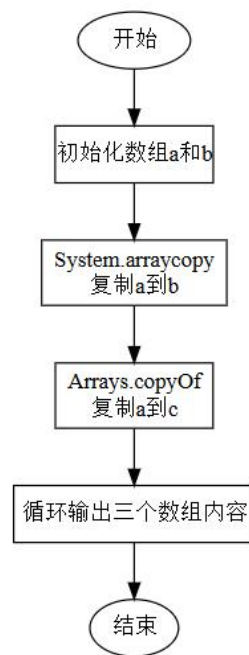


图 3 实现矩阵拷贝

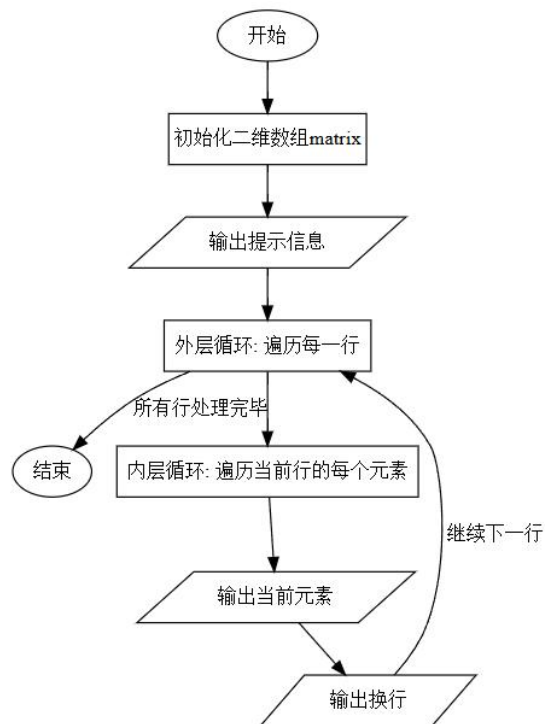


图 4 矩阵遍历

(2) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：我们需要利用 `Array` 的成员方法 `arraycopy` 与 `copyOf` 函数来实现数组复制，同时对于数组的遍历，我发现这非常类似 `python` 中的 `for` 语句，我们可以对照一下 `python` 中的 `for` 语句的语法来完成实验。

1) 编写 java 代码并运行

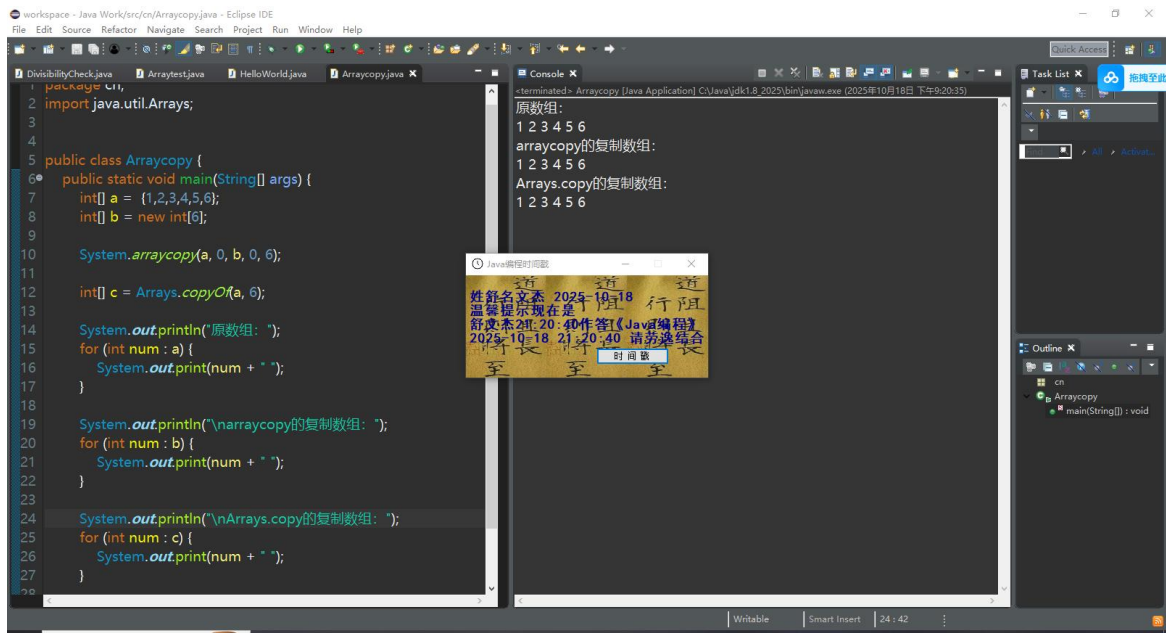


图 5 实验结果截图

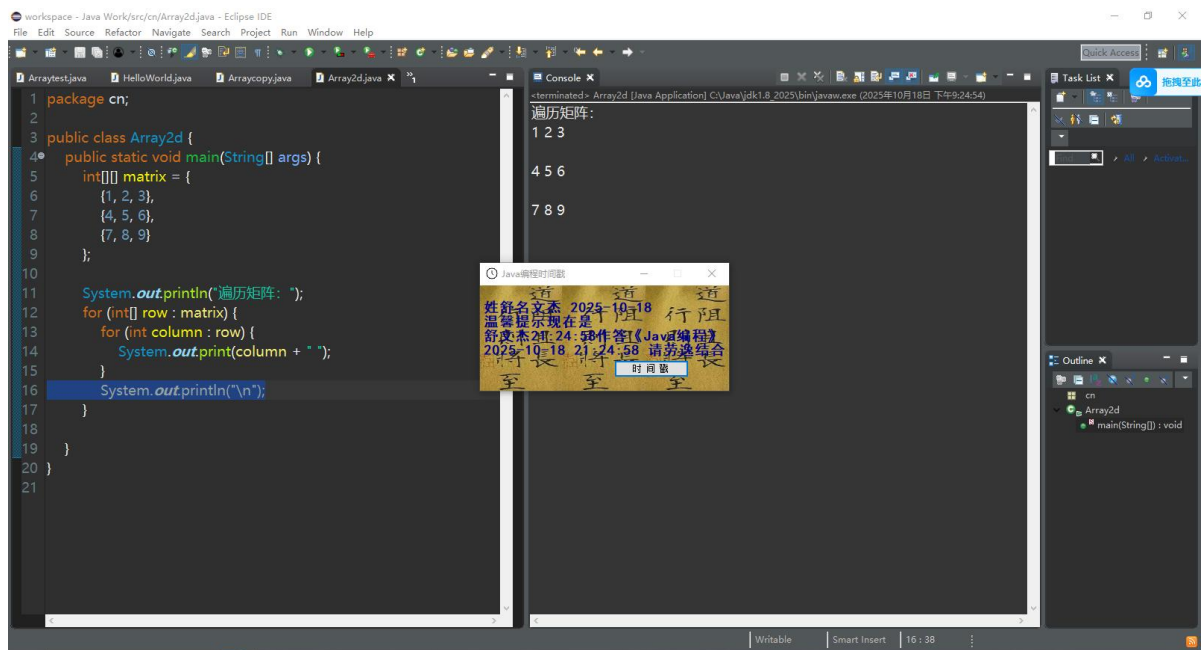


图 6 遍历矩阵结果

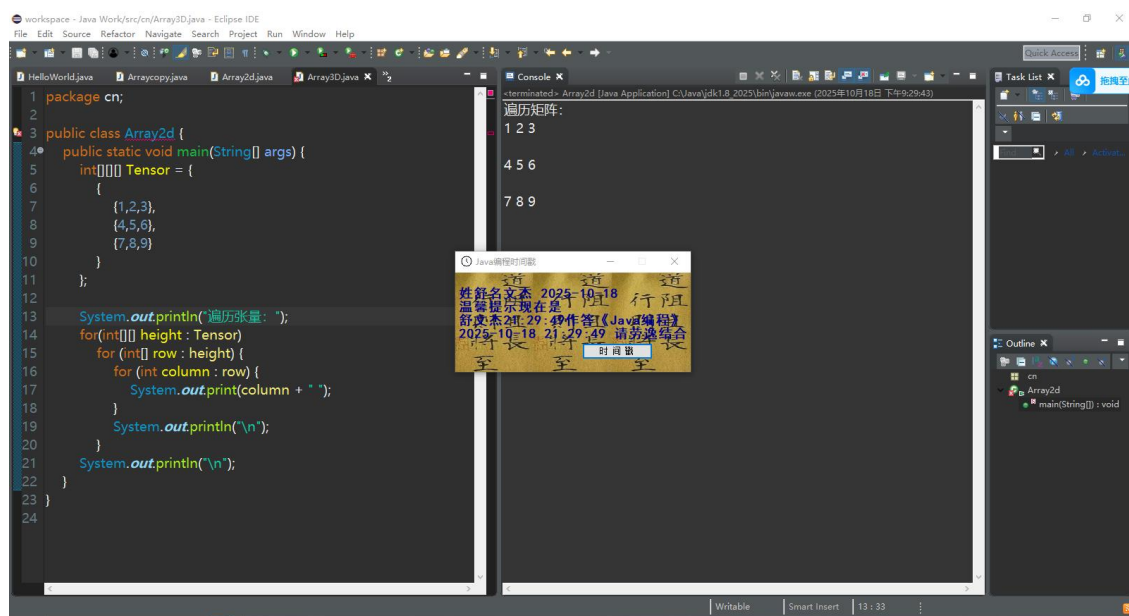


图 7 遍历张量结果

可以发现，我们的运行结果符合预期，我们的实验成功。

知识点总结：

数组部分：

1. 对于 Random 类的 nextInt()，其有两种重载方式，无参时生成全 int 范围内的随机整数，若有参数，则生成范围参数内的正整数。
2. arraycopy 与 copyof 均是实现数组拷贝的函数，但是它们之间也有共性，arraycopy 与 copyof() 均只是复制对象的地址，其并没有真正开辟空间，并且 copyof() 的底层实现是依靠 arraycopy 实现的。但是它们之间的区别在于 copyof() 会自动创建一个数组来返回实现数组的复制，但只能从 0 开始进行切片，arraycopy 更加细致，可以任意控制切片的开始与结束。
3. Java 中的 foreach 是以临时变量自动遍历目标的每个元素，无需手动管理索引，可避免索引越界问题；但需注意，遍历数组时变量是元素的副本，修改该变量不影响原数组，遍历集合时不能通过循环修改集合结构，且它无法获取元素索引、进行反向遍历或跳过元素，因此适用于仅需操作元素值、无需索引或修改结构的场景（如打印元素），若需索引、修改原数组，则需用普通 for 循环

3、编写一个程序，完成以下功能：

- 1) 声明一个名为 name 的 String 对象，内容是 “My name is NetworkCrazy”
- 2) 打印字符串的长度
- 3) 打印字符串的第一个字符
- 4) 打印字符串的最后一个字符

5) 打印子字符串 NetworkCrazy (substring() 方法提取子字符串)

6) 利用 lastIndexOf() 方法搜索字符 'e' 最后一次出现的位置

(1) 实验源程序

```
package cn;

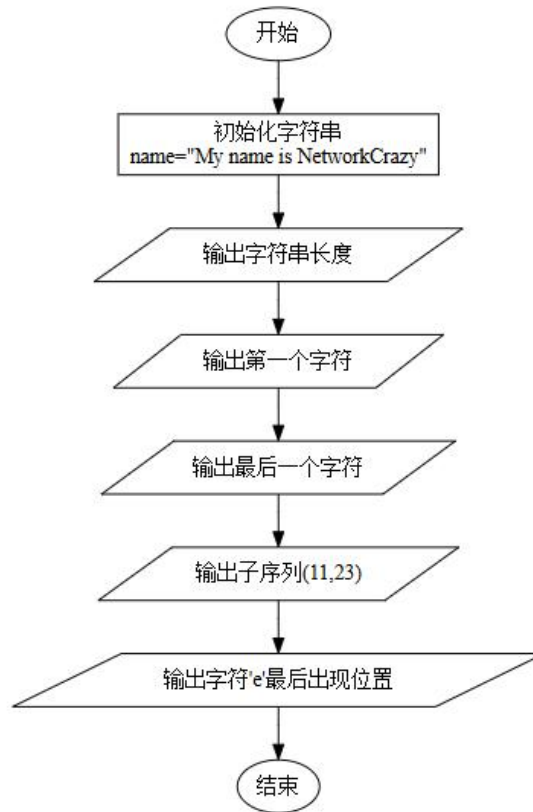
public class Stringtest {
    public static void main(String[] args) {
        String name = "My name is NetworkCrazy";
        System.out.print(name.length());

        System.out.println("\n 第一个字母是: ");
        System.out.print(name.charAt(0));
        System.out.println("\n 最后一个字母是: ");
        System.out.print(name.charAt(name.length()-1));

        System.out.println("\n 一个子序列为: ");
        System.out.print(name.substring(11,23));

        System.out.println("\n 字符 'e' 最后一次出现的位置: ");
        System.out.print(name.lastIndexOf('e'));
    }
}
```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：
实验代码思路：

1) 编写 java 代码并运行

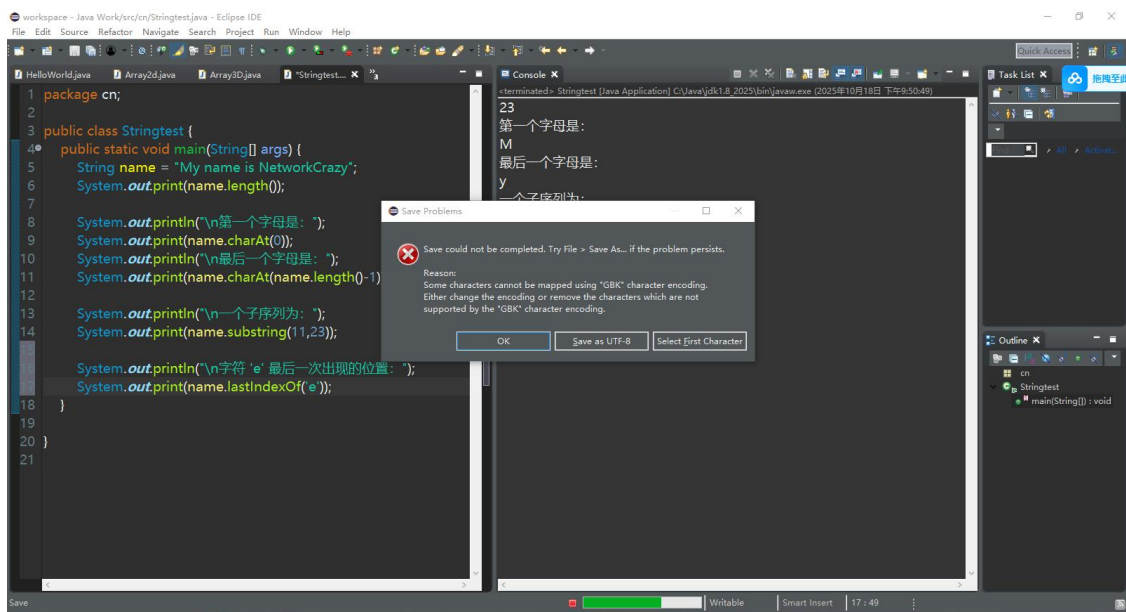


图 8 实验结果截图

？，为什么我的代码并没有明显的问题，会报如此奇怪的错误呢？后来发现，由于

我有一部分代码是在 vscode 中完成的，导致编码不一致，最后我重新保存解决了中国问题。

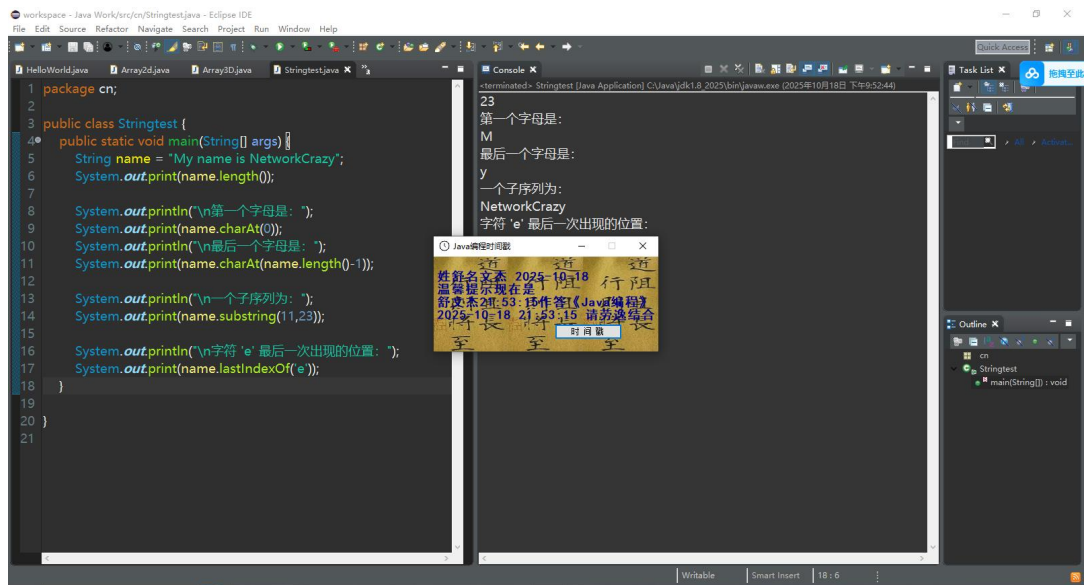


图 9 实验成功截图

我们的代码成功实现功能

4、利用 StringBuffer 类实现在字符缓冲区中存放字符串 "Happy new year!", 并通过 setCharAt 和 charAt 实现字符设置和替换, 利用 insert 实现字符的插入, 利用 append 实现在字符串末尾添加相应的内容。

(1) 实验源程序

```
package cn;

public class StringBuffertest {
    public static void main(String[] args) {
        StringBuffer stringbuffer = new StringBuffer("Happy new year!");

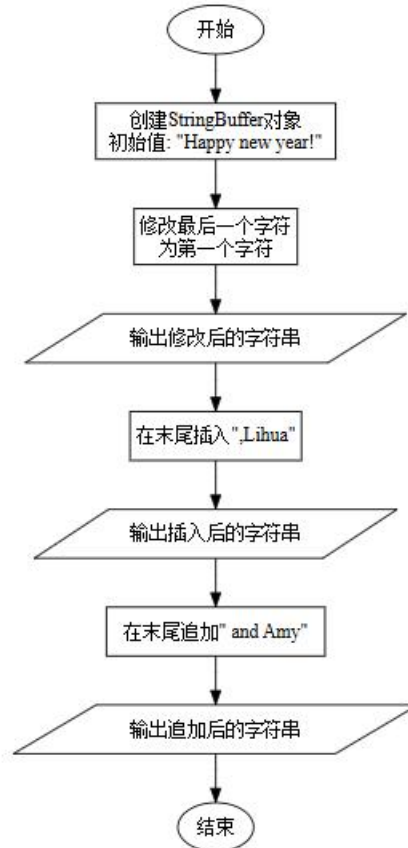
        System.out.println("\n 第一次变为: ");
        stringbuffer.setCharAt(stringbuffer.length()-1,
stringbuffer.charAt(0));
        System.out.print(stringbuffer);

        System.out.println("\ninsert 插入 Lihua: ");
        System.out.print(stringbuffer.insert(stringbuffer.length(),"Lihua"));

        System.out.println("\nappend 插入 and Amy: ");
        System.out.print(stringbuffer.append(" and Amy"));
    }
}
```

```
}
}
```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验思路：

1) 编写 java 代码

```
System.out.println("\n第一次插入Li: ");
stringbuffer.setCharAt(stringbuffer.length()-1,"L");
stringbuffer.setCharAt(stringbuffer.length() - 1, 'L');
```

图 10 参数不匹配

这里为什么两个几乎一样的代码，上面的报错是参数不匹配呢，我仔细观察后发现，原来是我使用了双引号导致的，这原本在 C 语言中，数据类型与单双引号是没有关系的，但是很明显，这在 java 中是有区别的。

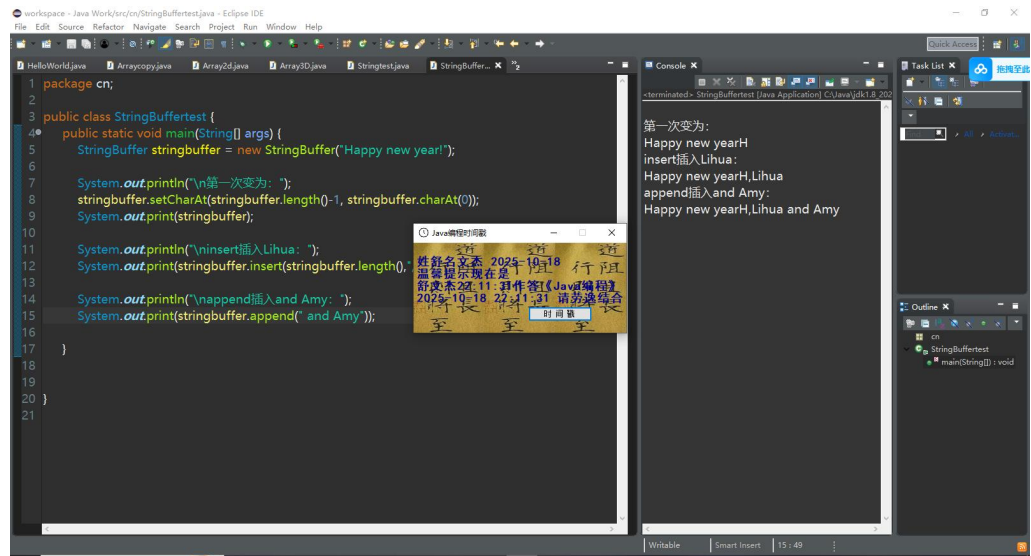


图 11 实验成功截图

可以发现，我们的运行结果符合预期，我们的实验成功。

知识点总结：

字符串：

1. `charAt (int index)` 方法：该方法的核心作用是返回字符串中指定索引位置的字符，字符串的索引遵循“从 0 开始”的规则（即第一个字符的索引为 0）。
2. `Substring` 方法：该方法有两种重载形式，第一种是 `substring(int beginIndex)`，其规则是从参数 `beginIndex` 开始，截取到字符串末尾，形成新的子串；第二种是 `substring(int beginIndex, int endIndex)`，规则是从 `beginIndex`（包含）截取到 `endIndex`（不包含该索引对应的字符）。
3. `Lastinstanceof` 方法：此方法的作用是查找指定字符在字符串中最后一次出现的索引，若字符串中不存在该字符，则返回-1。它是从字符串的末尾开始向前搜索，第一个匹配到指定字符的位置，即为该字符最后一次出现的索引，最终返回该索引值。

二、类与对象：

- 1、（1）创建一个 `Rectangle` 类，添加两个（private）属性 `width` 与 `height`

(2) 创建一个构造方法，选择 width 与 height 为形式参数。

(3) 在 Rectangle 类中添加两个方法计算矩形的周长和面积

(1) 实验源程序

```
package cn;

public class Geometrytest {
    public static void main(String[] args) {
        Rectangle tri = new Rectangle(3, 4);
        System.out.println("矩形面积: " + tri.getArea());
    }
}

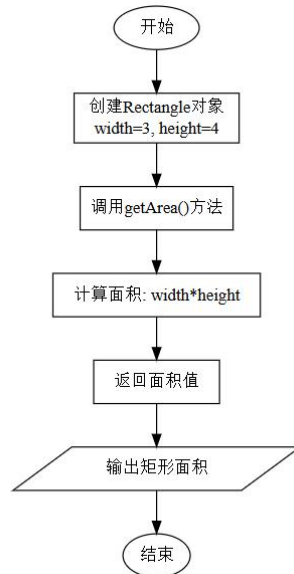
class Rectangle {

    private double width,height;
    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    public double getPerimeter() {
        return 2 * (width + height);
    }

    public double getArea() {
        return width*height;
    }
}
```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：由于我们需要构建一个具有私有属性的类构造其方法来实现计算，这里我们在外部无法直接调用私有属性，因此我们必须使用构造方法来实现构造器，将属性传入类中，因此我们需要直接使用公开的方法来返回面积。

1) 编写 java 代码

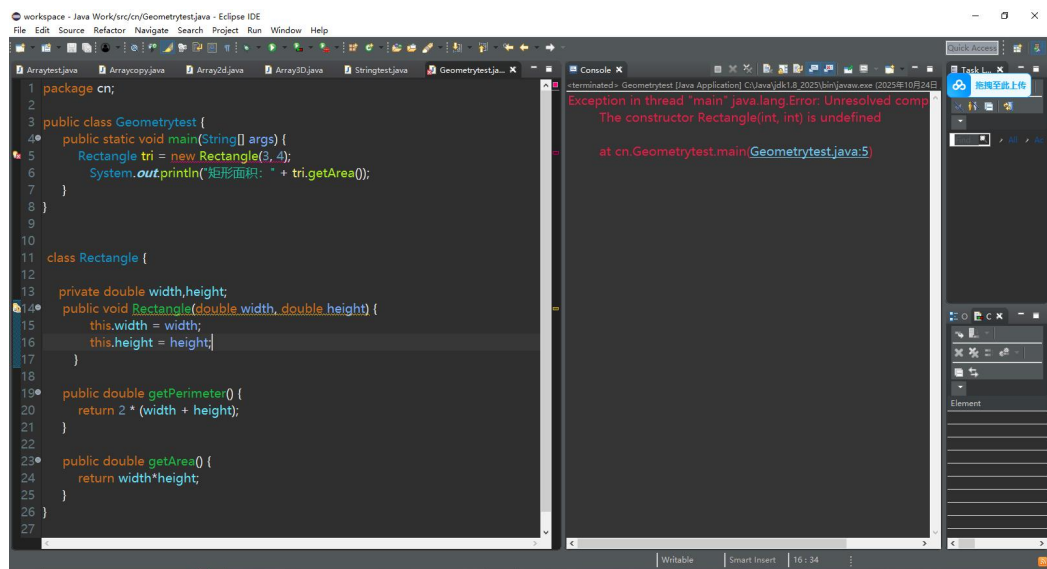


图 12 运行报错

我查阅资料后发现这里的报错是我实现构造方法时由于我的思维惯性，认为构造方法是一个不会返回值的方法，因此因此我们需要使用 `void` 进行修饰，后来我了解到，给构造方法加上 `void` 会令编译器认为它是一个普通方法从而报错，这是

整个 java 语言中唯一一个不需要显式声明 void 变量的地方。

```
package cn;

public class Geometry {

}

public class Rectangle {

    private double width,height;
    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    public double getPerimeter() {
        return 2 * (width + height);
    }

    public double getArea() {
        return width*height;
    }
}
```

图 13 类报错截图

这里我的主类被编译器报错，一开始我觉得很奇怪，后来在学习完修饰词的作用后发现，在一个 java 文件中，只能有一个类能被 public 修饰。因此得去掉 public。

2) 运行修改后的程序

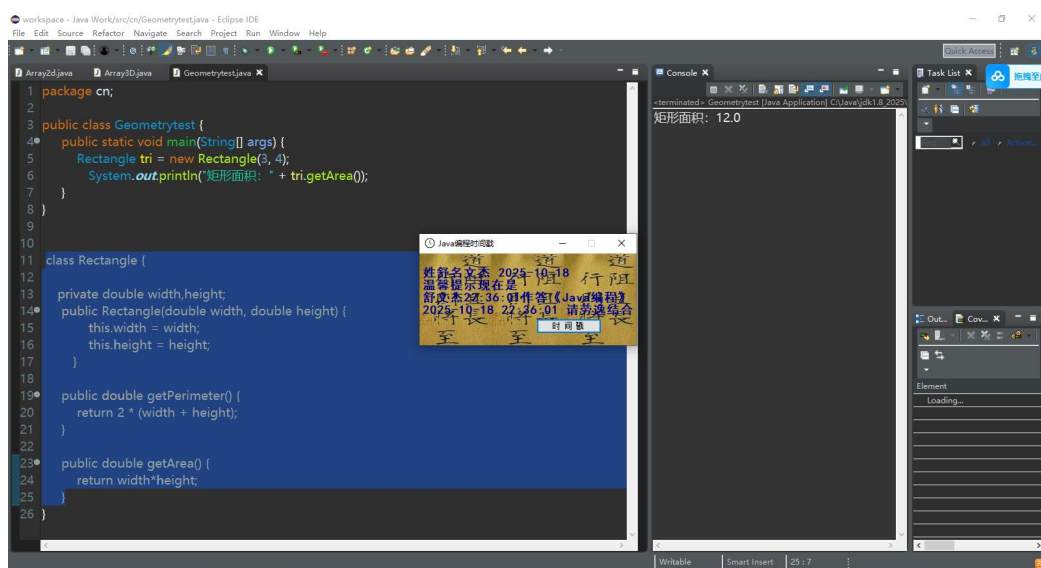


图 14 实验结果截图

可以发现，我们的实验成功了，这样便可以利用构造器为程序进行良好的封装。

2、创建三角形、梯形和圆形的类封装：分别定义三角形、梯形和圆形类，要求每个类中包含构造方法、求周长及面积的方法。最后在主方法 main 中为三角形、梯形和圆形类创建对象并打印输出各种图形的周长及面积。（一个文件中包含三个 class 和 main）

a) Trangle 类具有类型为 double 的三个边，以及周长、面积属性，Trangle 类具有返回周长、面积以及修改三个边的功能。另外，Trangle 类还具有一个 boolean 型的属性，该属性用来判断三个属能否构成一个三角形。

b) Lader 类有类型 double 的上底、下底、高、面积属性，具有返回面积的功能。

c) Circle 类具有类型为 double 的半径、周长和面积属性，具有返回周长、面积的功能。

(1) 实验源程序

```
package cn;

public class Geometry2 {
    public static void main(String[] args) {
        Triangle triangle= new Triangle(3,4,5);
        System.out.printf("三角形面积: %f;", triangle.area);
        System.out.printf("三角形周长: %f;", triangle.perimeter);
        triangle.setA(6.0, 8.0, 10.0);
        System.out.printf("\n 三角形面积: %f; ", triangle.area);
        System.out.printf("三角形周长: %f;", triangle.perimeter);
        //非法边长
        triangle.setA(1.0, 1.0, 10.0);
        System.out.printf("\n 三角形面积: %f; ", triangle.area);
        System.out.printf("三角形周长: %f;", triangle.perimeter);

        Lader lader = new Lader(2.0,4.0,5.0);
        System.out.printf("\n 梯形面积: %f;", lader.area);

        Circle circle = new Circle(4);
        System.out.printf("\n 圆形面积: %f;", circle.area);
        System.out.printf("圆形周长: %f;", circle.perimeter);
    }
}
```

```

}
class Triangle {
    private double a, b, c;
    public boolean isValid;
    public double area, perimeter;

    //利用构造方法获取属性
    public Triangle(double a, double b ,double c) {
        this.a = a;
        this.b = b;
        this.c = c;
        checkValid();
        getPerimeter();
        getArea();
    }

    //获取周长
    public void getPerimeter() {
        perimeter =( isValid)?(a+b+c) : 0;
    }

    //检查有效性
    public void checkValid() {
        isValid = (a + b > c) && (a + c > b) && (b + c > a);
    }

    //修改三边
    public void setA(double a,double b,double c) {
        if (a > 0 && b > 0 && c>0) {
            this.a = a;
            this.b = b;
            this.c = c;
            checkValid();
            getPerimeter();
            getArea();
        } else {
            System.out.println("边长必须为正数");
        }
    }

    //获取面积
    public void getArea() {
        double s = (a+b+c)/2;
        area = (isValid) ? (Math.sqrt(s * (s - a) * (s - b) * (s - c))) : 0;
    }
}
    
```

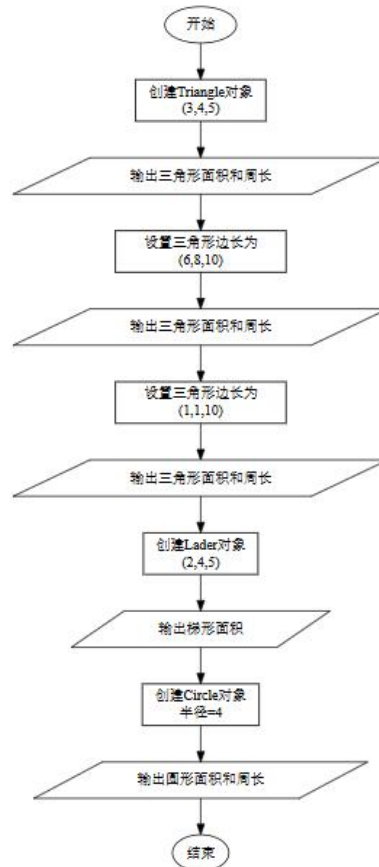
```

class Lader{
    private double top, bottem, height;
    public double area;
    //利用构造方法获取属性
    public Lader(double top, double bottem ,double height) {
        this.top = top;
        this.bottem = bottem;
        this.height = height;
        getArea();
    }
    //获取面积
    public void getArea() {
        area = (top + bottem)* height/2;
    }
}

class Circle{
    private double r;
    public double perimeter,area;

    //利用构造方法获取属性
    public Circle(double r) {
        this.r = r;
        getPerimeter();
        getArea();
    }
    //获取周长
    public void getPerimeter() {
        perimeter = 2*Math.PI*r;
    }
    //获取面积
    public void getArea() {
        area =Math.PI*r*r;
    }
}
    
```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：在这次实验中，要求我们需要定义一系列的类属性，同时，为了返回由输入属性计算出来的结果能够得到，我们将面积等属性使用 `public` 修饰，因此主类中我们便可以通过 `.` 运算符引出所得到的属性，需要注意的是在构造方法里面我们需要调用对应的方法来计算属性；

对于特殊的三角形，由于我们不知道能否构成三角形，因此，我们应该先计算 `isvalid` 属性再判断是否进行下一步计算。

1) 编写 java 代码并运行

```

    }

    private void checkValid() {
        isvalid = (a + b > c) && (a + c > b) && (b + c > a);
    }

    public double getArea() {
        double s = (a + b + c) / 2;
        double area = Math.sqrt(s * (s - a) * (s - b) * (s - c));
        return area;
    }
    
```

图 15 编译器报错

这里发生了报错，由于我们的 `isvalid` 是一个共有的属性，因此我们使用 `private` 修饰是完全不合法的，需要进行修改。

2) 运行修改后的程序

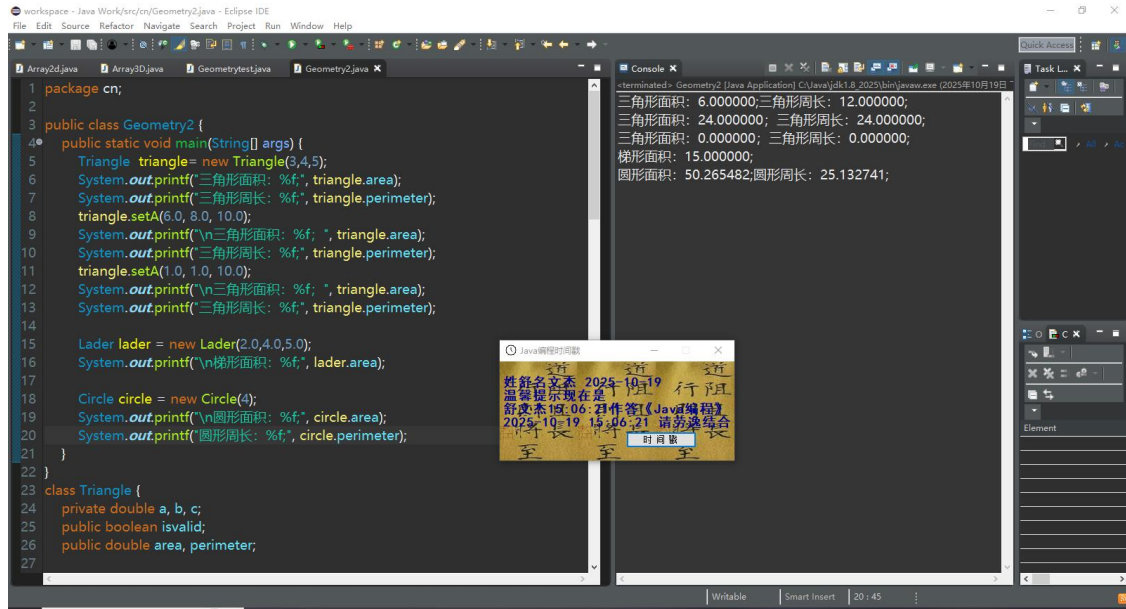


图 16 实验结果截图

可以发现，我们可以正确地完成修改三边计算面积，同时对于非法的三角形，我们将其周长与面积归零，因此以上的实验结果符合预期，我们的实验成功。

3、方法重载的练习

编写一个程序，定义一个类，类有 3 个 `plus()` 方法，分别实现两个数相加，三个数相加，四个数相加的程序。

(1) 实验源程序

```

package cn;

public class Caculate {
    public static void main(String[] args) {
        Caculateplus plus = new Caculateplus();
        System.out.printf("两数相加:%d", plus.plus(1, 1));
        System.out.printf("\n 三数相加:%d", plus.plus(1, 1, 1));
        System.out.printf("\n 四数相加:%d", plus.plus(1, 1, 1, 1));
    }
}

class Caculateplus {
    public int plus (int a ,int b) {
    
```

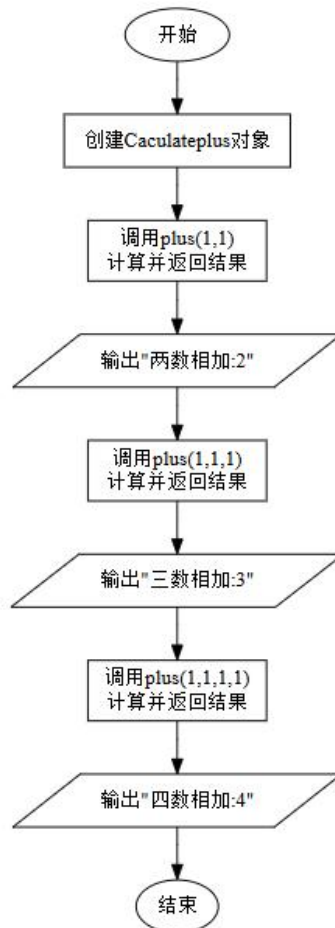
```

        return a+b;
    }

    public int plus (int a ,int b ,int c) {
        return a+b+c;
    }

    public int plus (int a ,int b ,int c ,int d ) {
        return a+b+c +d;
    }
}
    
```

(2) 实验流程图：



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：这个实验是我们了解与练习重载的大好时机。在课上听得有些云里雾里，在这里我们便可以进行实践。根据定义，我们需要在同一类中构造多个方法名一致但参数类型或个数不一致的方法，那么我们便可以根据此思路进行实验。

1) 编写 java 代码并运行

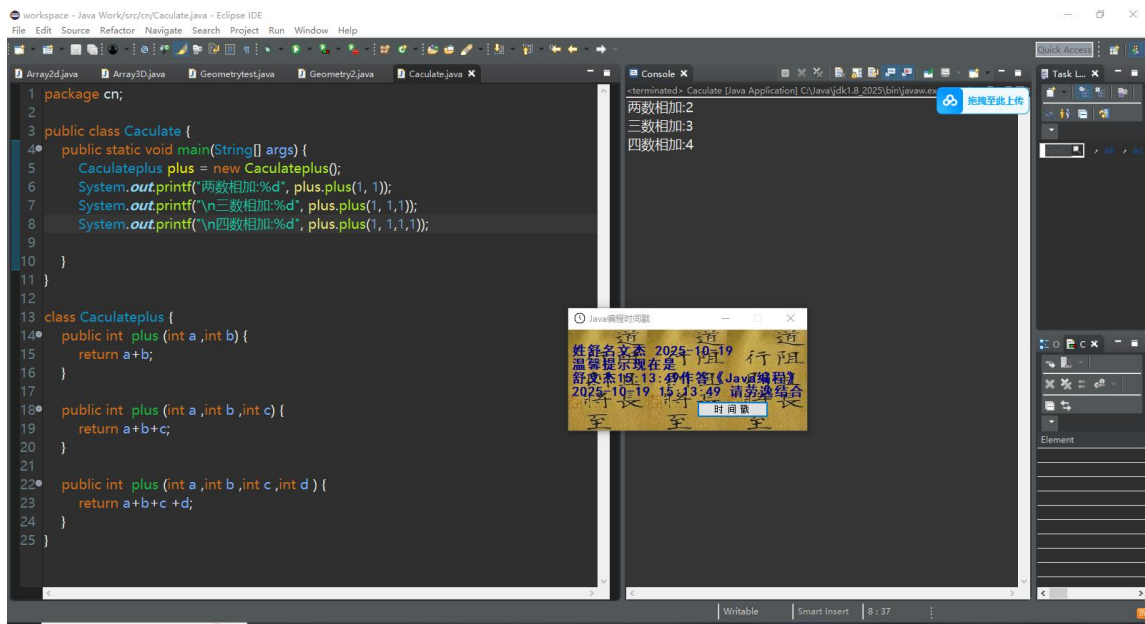


图 17 实验结果截图

可以发现，方法的重载被我们很简单的实现了，这种形式的方法让我想起了 Matlab 中的很多函数也是方法名一致但传入的参数和类型不一致，看来是同一种形式实现的吧。

4、编译并运行下面的程序，观察分析运行结果，体会程序 super 和 this 的用法，进一步理解变量隐藏和方法重写的概念。

(1) 实验源程序

```
package cn;

class Person {
    int weight;
    Person() {
        weight = 50;
    }
    void printPerson() {
        System.out.println("Person.printPerson()");
    }
}

class ZhangSan extends Person {
    int weight;
    ZhangSan() {
        super();
    }
}
```

```

        weight = 500;
    }
    void printPerson() {
        System.out.println("ZhangSan.printPerson()");
    }
    void superThisUseDemo() {
        int weight;
        weight = 5000;
        super.printPerson();
        printPerson();
        System.out.println("super.weight=" + super.weight);
        System.out.println("this.weight=" + this.weight);
        System.out.println("weight=" + weight);
    }
}

public class SuperThisUseDemo {
    public static void main(String[] args) {
        ZhangSan zhangsan = new ZhangSan();
        zhangsan.superThisUseDemo();
    }
}

```

(2) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

1) 编写 java 代码并运行

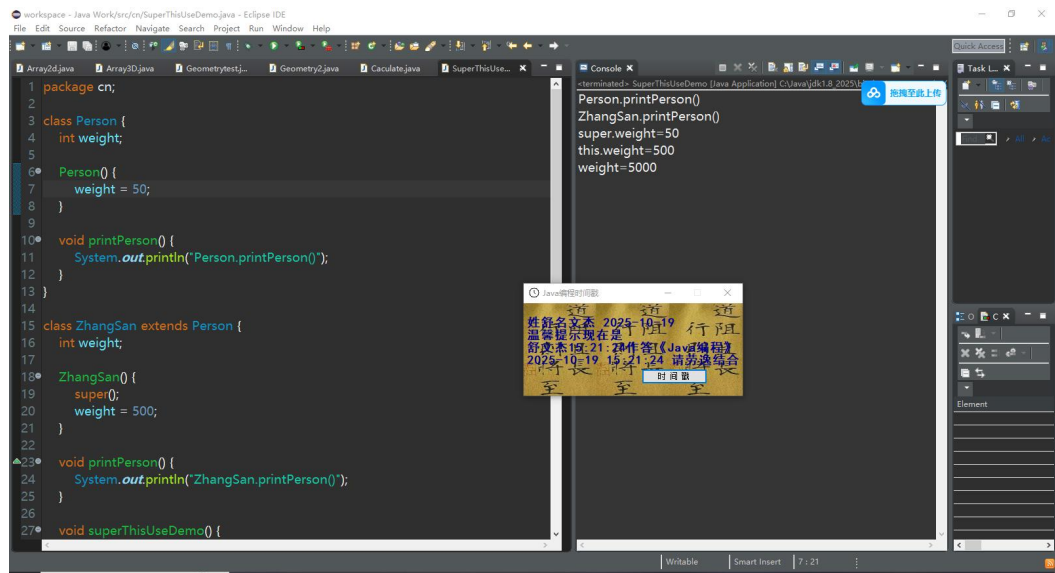


图 18 运行结果截图

我们来观察一下实验输出的结果，很明显，我们可以发现对应使用 `super` 点出的方法是 `ZhangSan` 类的父类的成员方法，而 `this` 点出的是 `ZhangSan` 自身的成员方

法，那么 `super` 与 `this` 的用法我们便可以得到初步总结了。

在 `Zhansan` 中，有一个与父类同名的方法 `printPerson()`，且方法参数、返回值完全一致，这符合重写的定义，因此子类张三本地的方法重写了父类的 `printPerson()` 方法，调用 `printPerson()` 只能得到子类重写的方法

我们 `weight` 中的输出结果分别是 50, 500, 5000，这分别对应了父类的成员属性，子类的成员属性，以及本地的局部属性，这表示，在同一个方法中，一个属性可以有三个值，这更加体现了老师所说 Java 是一个立体的语言，这一层层变量便是变量隐藏，变量隐藏使得子类与父类的耦合减少，便不容易出现 C 语言那样改一个函数，报了上百个 `error` 的情况。

(3) 实验总结：

1. `super` 与 `this` : 在子类中, `super.方法名()` 用于调用父类的成员方法, `this.方法名()` (或直接调用) 用于调用子类自身的成员方法, 二者明确区分了继承关系中不同层级的方法调用。

2. 方法重写: 当子类存在与父类同名、同参数、同返回值的方法时, 构成方法重写; 调用该方法时, 会优先执行子类重写后的版本, 体现了子类对父类方法的定制化实现。

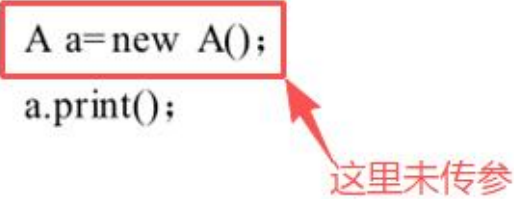
3. 变量隐藏: 同名变量可存在于父类 (成员变量)、子类 (成员变量)、方法局部 (局部变量) 中, 通过 `super`、`this` 和直接访问可分别获取对应层级的值, 形成变量隐藏; 这种机制减少了子类与父类的耦合, 提升了代码的独立性。

5、下面的代码有什么错误，应该如何解决？

```

1  class Test {
2      public static void main(String[] args){
3          A a=new A();
4          a.print();
5      }
6  }
7  class A {
8      String s;
9      A(String s){
10         this.s=s;
11     }
12     public void print(){
13         System.out.println(s);
14     }
15 }

```



(1) 问题解决:

由我上图标出，很明显，这里代码的错误是明明在 A 中的构造方法明明要求传入属性 s 但是在主类中引用时并没有进行传入，则必然会报错。

因此，有两种解决方法，其一是从主类传入字符串，如此便可以解决报错。

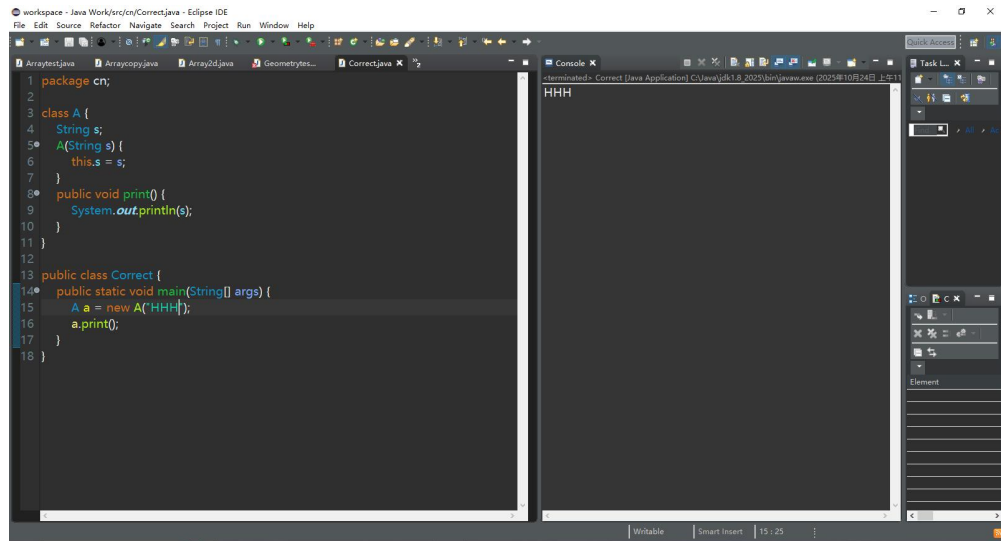


图 19 为主类传入字符串

这是相当简单的一种改错方法，但是如果我们不想修改主类，我们便可以试试对于构造方法进行重载。

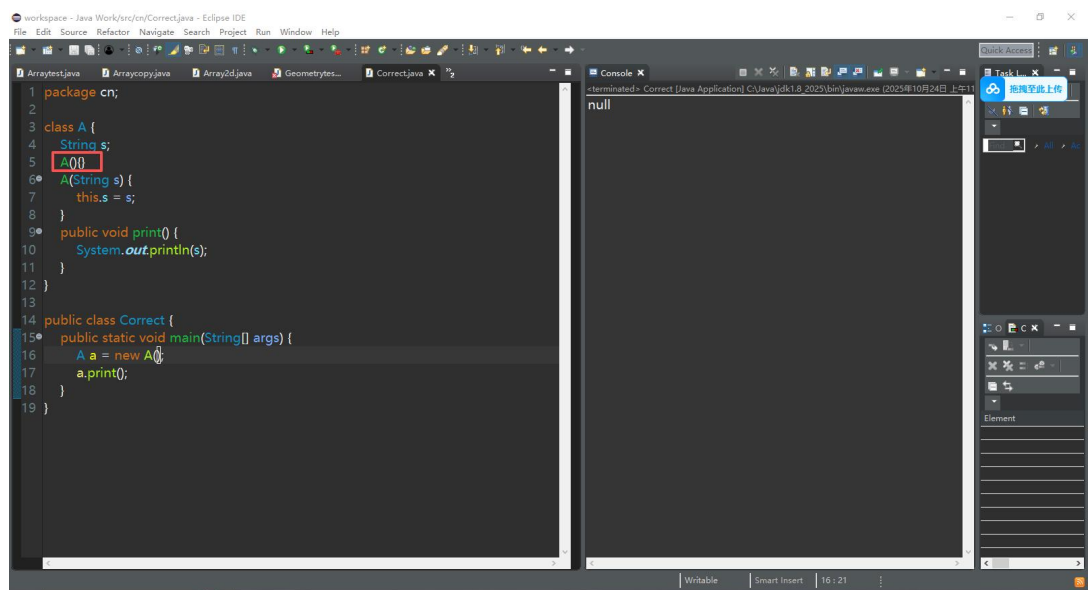


图 20 重载构造方法

可以发现，对于构造方法进行重载的结果也成功了，说明构造方法也是可以重载的。

知识点总结：类与对象

1. 构造方法

在 Java 中，构造方法是类初始化的核心机制，它无返回值且绝对不能用 void 修饰，若错误添加 void（如实验 1 中对构造器的误操作），编译器会将其判定为普通方法，从而触发编译错误。当类中定义了带参构造器时，Java 不会再自动生成无参

构造器，这会导致无参创建对象的操作失败（如实验 5 中类 A 仅存在带参构造器 A(String s)，new A() 因无匹配构造器报错），此时可通过构造器重载解决——为类新增无参构造器，使其与原有带参构造器形成重载关系。此外，一个 Java 源文件中仅能有一个类被 public 修饰，若多个类同时被 public 修饰（如实验 1 中主类与 Rectangle 类的不当修饰），会直接触发编译错误。

2. 方法的重载与重写

方法重载与方法重写是 Java 方法体系中的关键概念。方法重载指在同一类中定义多个方法名相同但参数列表不同（参数个数、类型或顺序存在差异）的方法，其与返回值类型无关（如实验 3 中 Caculateplus 类的 3 个 plus() 方法，分别接收 2 个、3 个、4 个 int 参数，实现不同个数的加法操作，调用时会根据传入参数自动匹配对应方法）。方法重写则发生在继承关系中，子类需定义与父类“方法名、参数列表、返回值”完全一致的方法，以此实现对父类方法逻辑的定制化修改；执行时，子类对象会优先调用自身重写的方法，若需调用父类原方法，需通过 super. 方法名() 显式指定（如实验 4 中 ZhangSan 类重写了 Person 类的 printPerson() 方法，直接调用 printPerson() 执行子类逻辑，而 super.printPerson() 则执行父类逻辑）。

6. 熟悉 Calendar 类和 Date 类的成员方法，完成万年历的功能

(1) 实验源程序

```
package cn;

import java.awt.*;
import javax.swing.*;
import java.awt.event.*;
import java.util.Calendar;

public class Calendar1 extends JFrame implements ActionListener {
    private static final long serialVersionUID = 1072582448511199623L;
    private JTextField yearField, monthField;
    private JButton Button;
    private JTextArea calendarArea;

    public static void main(String[] args) {
        new Calendar1();
    }

    public Calendar1(){
        init();
        this.setSize(700, 300);
        this.setVisible(true);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

```

//初始化函数
public void init() {
    //生成面板
    JPanel mainPanel = new JPanel(new BorderLayout());
    JPanel inputPanel = new JPanel();
    //年份输入面板
    JLabel labely = new JLabel("年份:");
    inputPanel.add(labely);
    yearField = new JTextField(5);
    inputPanel.add(yearField);
    //月份输入面板
    JLabel labelm = new JLabel("月份(1-12):");
    inputPanel.add(labelm);
    monthField = new JTextField(3);
    inputPanel.add(monthField);
    //查询按钮
    Button = new JButton("查询");
    Button.addActionListener(this);
    inputPanel.add(Button);
    //将面板添加到中央
    mainPanel.add(inputPanel, BorderLayout.NORTH);
    //添加月份显示面板
    calendarArea = new JTextArea();
    calendarArea.setEditable(false);
    JScrollPane scrollPane = new JScrollPane(calendarArea);
    mainPanel.add(scrollPane, BorderLayout.CENTER);
    //添加主面板
    this.add(mainPanel);

    showCurrentCalendar();
}

private void showCurrentCalendar() {
    Calendar calendar = Calendar.getInstance();
    int year = calendar.get(Calendar.YEAR);
    int month = calendar.get(Calendar.MONTH) - 1 ;
    showCalendar(year, month);
}

private void showCalendar(int year, int month) {
    Calendar calendar = Calendar.getInstance();
    calendar.set(year, month-1, 1);

    StringBuilder stringbuilder = new StringBuilder();

```

```

        stringBuilder.append(year).append("年").append(month).append("月\n");
        stringBuilder.append("日\t一\t二\t三\t四\t五\t六\n");

        int firstDayOfWeek = calendar.get(Calendar.DAY_OF_WEEK);
        int preDaysCount = firstDayOfWeek - 1;

        int daysInCurrentMonth =
calendar.getActualMaximum(Calendar.DAY_OF_MONTH);

        int preyear = year;
        int preCalMonth = month - 2;
        if(preCalMonth == 0) {
            preyear = preyear - 1;
            preCalMonth = 11;
        }
        calendar.set(preyear, preCalMonth, 1);
        int daysInPreMonth = calendar.getActualMaximum(Calendar.DAY_OF_MONTH);

        int preShowDay = daysInPreMonth - preDaysCount + 1;
        for(int i=0; i < preDaysCount; i+=1) {
            stringBuilder.append(' ').append(preShowDay).append("]\t");
            preShowDay+=1;
        }

        int currentShowDay = 1;
        for(int i=0; i < daysInCurrentMonth; i+=1) {
            stringBuilder.append(currentShowDay).append('\t');
            if ((preDaysCount + currentShowDay) % 7 == 0) {
                stringBuilder.append("\n");
            }
            currentShowDay+=1;
        }

        int nextFillDay = 7-(preDaysCount + daysInCurrentMonth)%7;
        //小于一行才要填充
        if(nextFillDay < 7) {
            int nextyear = year;
            int nextmonth = month ;

            if(nextmonth == 12) {
                nextyear = year + 1;
                nextmonth = 0;
            }
            calendar.set(nextyear, nextmonth, 1);
        }
    
```

```

        int nextShowDay = 1;
        for(int i=0; i < nextFillDay; i+=1) {
            stringBuilder.append(' ').append(nextShowDay).append("\t");
            nextShowDay+=1;
        }

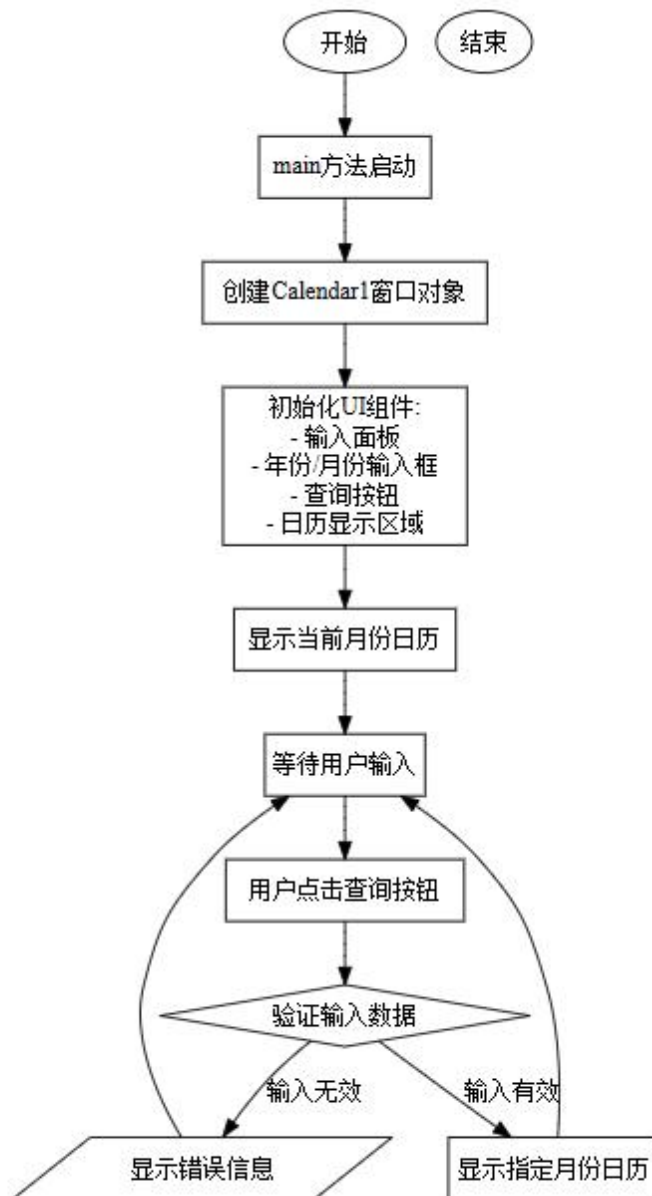
    }

    calendarArea.setText(stringbuilder.toString());
}

/** 处理按钮点击事件（查询逻辑） */
@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == Button) {
        try {
            // 解析输入的年份和月份
            int year = Integer.parseInt(yearField.getText().trim());
            int month = Integer.parseInt(monthField.getText().trim());
            // 异常处理
            if (month < 1 || month > 12) {
                JOptionPane.showMessageDialog(this, "月份必须在 1-12 之间", "
输入错误", JOptionPane.ERROR_MESSAGE);
                return;
            }
            // 显示日历
            showCalendar(year, month);
        } catch (NumberFormatException ex) {
            // 异常处理
            JOptionPane.showMessageDialog(this, "请输入有效的数字", "输入错误",
JOptionPane.ERROR_MESSAGE);
        }
    }
}
}

```

(2) 实验流程图：



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验思路简述：同上次实验一致，我们可以通过 Java Swing 构建可视化日历查询工具，采用 Swing 组件设计界面；我们需要设定的即通过 showCalendar 方法计算指定年月的日历布局。在其中，我们需要确定当月第一天星期几、前后月天数，最后拼接包含年月标题、星期表头、上月日期（[] 标注）、当月日期及下月填充日期的文本（这部分可以使用我们本次实验所学的. append 函数来实现）；同时，不要忘记异常出理，对于验证用户输入的年月合法性（月份 1-12，输入为有效数字）进行验证。

1) 编写 java 代码

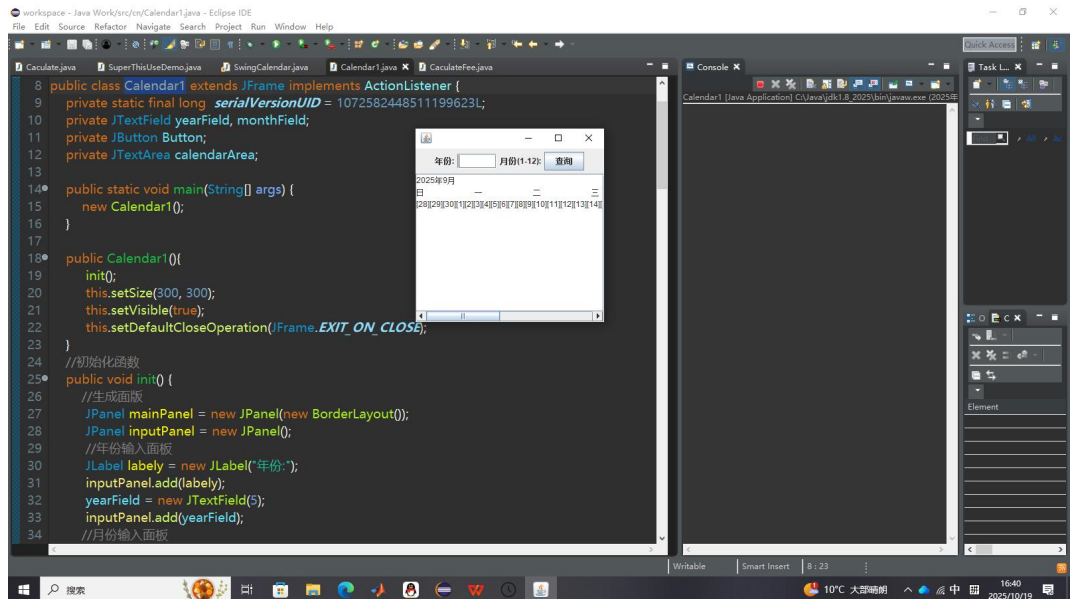


图 21 排版错误

可以发现，在实现日历排布时，我们可能忘记了对于每一个星期进行换行处理，而且，我们的月份输入面板也不见了，这明显是我编写过程中出现问题：

```
//初始化函数
public void init() {
    //生成面板
    JPanel mainPanel = new JPanel(new BorderLayout());
    JPanel inputPanel = new JPanel();
    //年份输入面板
    JLabel labely = new JLabel("年份:");
    inputPanel.add(labely);
    yearField = new JTextField(5);
    inputPanel.add(yearField);
    //月份输入面板
    JLabel labelm = new JLabel("月份(1-12):");
    inputPanel.add(labelm);
    monthField = new JTextField(3);
    //查询按钮
    Button = new JButton("查询");
```

← 忘记添加到面板上了

图 22 添加面板

```

int preCalMonth = month - 1;
if(preCalMonth == 0) {
    preyear = preyear - 1;
    preCalMonth = 12;
}
calendar.set(preyear, preCalMonth, 1);
int daysInPreMonth = calendar.getActualMaximum(Calendar.DAY_OF_MONTH);

int preShowDay = daysInPreMonth - preDaysCount + 1;
for(int i=0; i < preDaysCount; i+=1) {
    stringBuilder.append("").append(preShowDay).append("\t");
    preShowDay++;
}

int currentShowDay = 1;
for(int i=0; i < daysInCurrentMonth; i+=1) {
    stringBuilder.append("").append(currentShowDay).append("\t");
    if ((preDaysCount + currentDay) % 7 == 0) {
        stringBuilder.append("\n");
    }
    currentShowDay++;
}

int nextFillDay = 7-(preDaysCount + daysInCurrentMonth)%7;
//小于一行要填充
if(nextFillDay < 7) {
    int nextyear = year;
}
    
```

每七天换行一次

图 23 实现换行逻辑

2) 修改代码后，重复执行

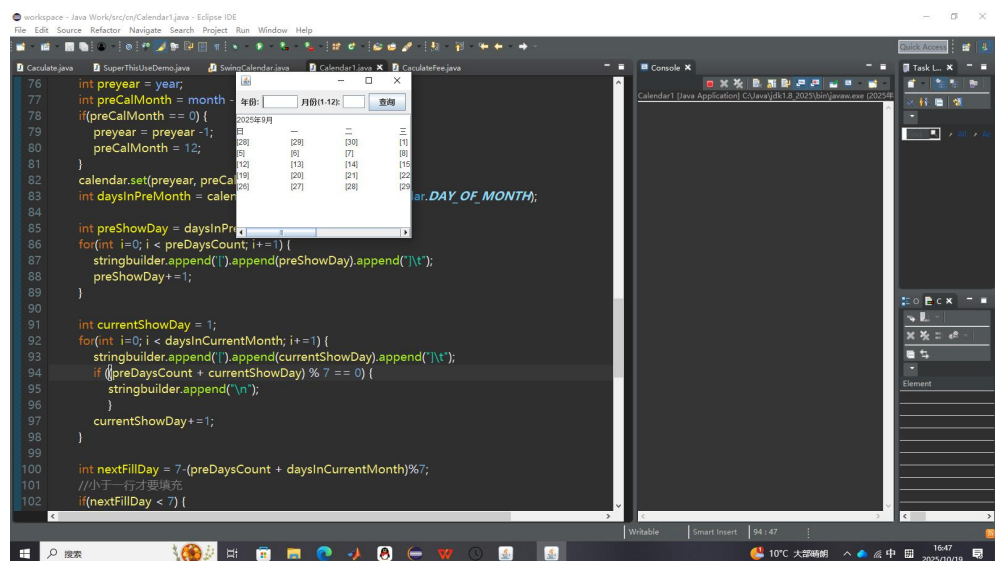


图 24 面板过小且添加括号逻辑混乱

可以发现，我们套用上一次实验的模板进行的初始化布局的面板过小，同时，我们的我们的括号添加逻辑存在很大的问题。

3) 修改面板布局等问题后，再次执行程序

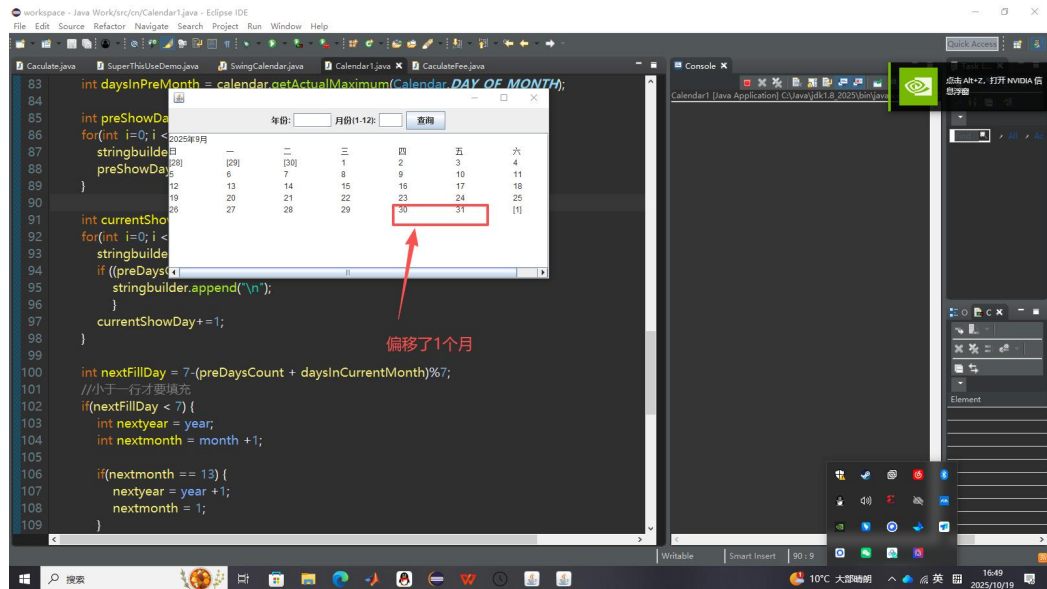


图 25 万年历发生偏移

我们发现我们的万年历的日期对不上正确的日期，且恰好便宜了一个月，仔细检查相关代码后发现问题在于我对 *Calendar* 类理解存在问题，它的 *month* 是从 0 开始，而我们的是从 1 开始计算，因此所有的 *month* 都应该减去 1。

```
int year = calendar.get(Calendar.YEAR);
int month = calendar.get(Calendar.MONTH) - 1;
```

4) 修改完成后，再次运行程序

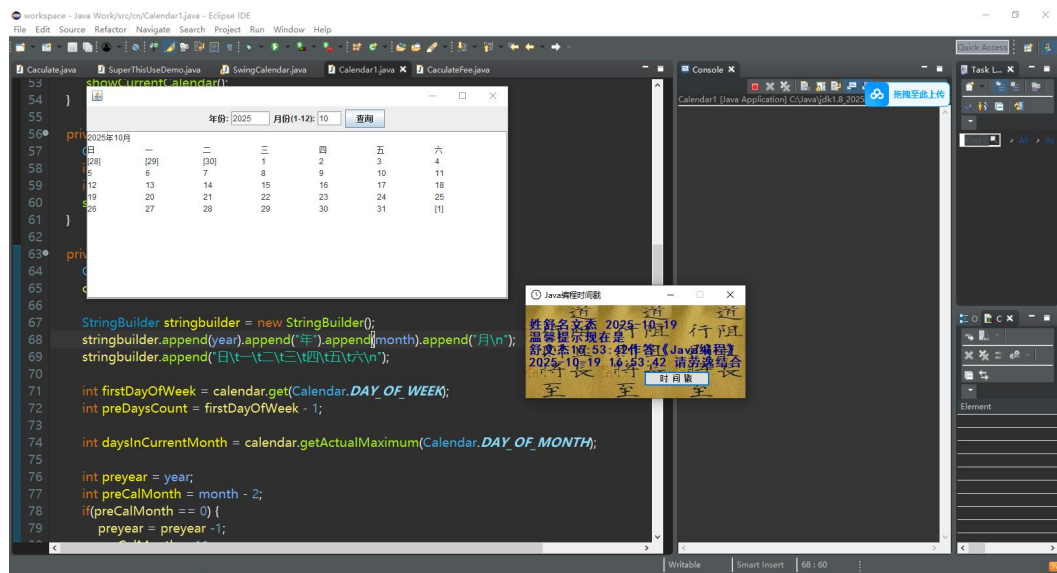


图 26 实验成功截图

可以发现，我们的实验完全成功。

Update:

更新后代码：

```

package cn;

import java.awt.*;
import java.awt.event.*;
import java.util.Calendar;
import java.util.GregorianCalendar;
import javax.swing.*;
import javax.swing.border.LineBorder;

public class Calendar1 extends JFrame implements ActionListener {
    private static final long serialVersionUID = 1L;

    // UI 组件
    private JLabel currentMonthLabel; // 显示当前年月
    private JPanel calendarPanel;    // 日历网格区域
    private JButton prevMonthBtn, nextMonthBtn, todayBtn;
    private JSpinner yearSpinner, monthSpinner; // 改用微调器输入

    // 数据
    private Calendar currentCal; // 当前显示的日期对象

    public static void main(String[] args) {
        // 使用 SwingUtilities 确保线程安全
        SwingUtilities.invokeLater(() -> new Calendar1());
    }

    public Calendar1() {
        // 初始化日历对象为当前时间
        currentCal = Calendar.getInstance();

        initUI();
        refreshCalendar(); // 首次渲染

        this.setTitle("Java 万年历 - 优化版");
        this.setSize(600, 500);
        this.setLocationRelativeTo(null); // 居中显示
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        this.setVisible(true);
    }

    private void initUI() {
        this.setLayout(new BorderLayout(10, 10));

        // --- 1. 顶部控制面板 ---
    }

```

```

JPanel topPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 10));
topPanel.setBackground(new Color(240, 240, 240));

prevMonthBtn = createButton("<< 上月", "PREV");
nextMonthBtn = createButton("下月 >>", "NEXT");
todayBtn = createButton("回到今天", "TODAY");

// 年份微调器 (1900 - 2100)
yearSpinner = new JSpinner(new
SpinnerNumberModel(currentCal.get(Calendar.YEAR), 1900, 2100, 1));
JSpinner.NumberEditor yearEditor = new JSpinner.NumberEditor(yearSpinner,
"#");
yearSpinner.setEditor(yearEditor);
yearSpinner.setPreferredSize(new Dimension(80, 30));
yearSpinner.addChangeListener(e -> updateFromSpinners());

// 月份微调器 (1 - 12)
monthSpinner = new JSpinner(new
SpinnerNumberModel(currentCal.get(Calendar.MONTH) + 1, 1, 12, 1));
monthSpinner.setPreferredSize(new Dimension(60, 30));
monthSpinner.addChangeListener(e -> updateFromSpinners());

topPanel.add(prevMonthBtn);
topPanel.add(new JLabel("年:"));
topPanel.add(yearSpinner);
topPanel.add(new JLabel("月:"));
topPanel.add(monthSpinner);
topPanel.add(nextMonthBtn);
topPanel.add(todayBtn);

this.add(topPanel, BorderLayout.NORTH);

// --- 2. 中央日历面板 ---
// 包含星期头和日期网格
JPanel centerPanel = new JPanel(new BorderLayout());

// 星期标题头
JPanel weekHeaderPanel = new JPanel(new GridLayout(1, 7));
weekHeaderPanel.setBackground(new Color(230, 230, 250));
String[] weeks = {"日", "一", "二", "三", "四", "五", "六"};
for (String w : weeks) {
    JLabel label = new JLabel(w, JLabel.CENTER);
    label.setFont(new Font("微软雅黑", Font.BOLD, 14));
    if ("日".equals(w) || "六".equals(w)) {

```

```

        label.setForeground(Color.RED);
    }
    label.setBorder(BorderFactory.createEmptyBorder(5, 0, 5, 0));
    weekHeaderPanel.add(label);
}
centerPanel.add(weekHeaderPanel, BorderLayout.NORTH);

// 日期网格 (6 行 7 列)
calendarPanel = new JPanel(new GridLayout(6, 7, 2, 2));
calendarPanel.setBackground(Color.WHITE);
calendarPanel.setBorder(BorderFactory.createEmptyBorder(5, 5, 5, 5));
centerPanel.add(calendarPanel, BorderLayout.CENTER);

this.add(centerPanel, BorderLayout.CENTER);

// --- 3. 底部信息栏 ---
JPanel bottomPanel = new JPanel();
currentMonthLabel = new JLabel("");
currentMonthLabel.setFont(new Font("微软雅黑", Font.BOLD, 16));
bottomPanel.add(currentMonthLabel);
this.add(bottomPanel, BorderLayout.SOUTH);
}

private JButton createButton(String text, String command) {
    JButton btn = new JButton(text);
    btn.setActionCommand(command);
    btn.addActionListener(this);
    btn.setFocusPainted(false);
    btn.setBackground(Color.WHITE);
    return btn;
}

/**
 * 核心逻辑：刷新日历显示
 */
private void refreshCalendar() {
    calendarPanel.removeAll(); // 清除旧的日期

    int year = currentCal.get(Calendar.YEAR);
    int month = currentCal.get(Calendar.MONTH); // 0-11

    // 更新底部标签和 Spinner 状态（防止循环触发）
    currentMonthLabel.setText(String.format("当前显示: %d 年 %d 月", year,
month + 1));

```

```

yearSpinner.setValue(year);
monthSpinner.setValue(month + 1);

// 计算当月第一天
Calendar cal = (Calendar) currentCal.clone();
cal.set(Calendar.DAY_OF_MONTH, 1);

// 获取当月第一天是星期几 (1=周日, 2=周一 ... 7=周六)
int startDayOfWeek = cal.get(Calendar.DAY_OF_WEEK);

// 将日期回拨到日历面板的起始位置 (即上个月的某些天)
// 如果 1 号是周日(1), 回拨 0 天; 如果是周一(2), 回拨 1 天...
cal.add(Calendar.DAY_OF_MONTH, -(startDayOfWeek - 1));

// 获取“今天”的真实日期, 用于高亮
Calendar realToday = Calendar.getInstance();

// 循环填充 6 行 * 7 列 = 42 个格子
for (int i = 0; i < 42; i++) {
    int dy = cal.get(Calendar.YEAR);
    int dm = cal.get(Calendar.MONTH);
    int dd = cal.get(Calendar.DAY_OF_MONTH);

    JLabel dayLabel = new JLabel(String.valueOf(dd), JLabel.CENTER);
    dayLabel.setOpaque(true); // 允许设置背景色
    dayLabel.setFont(new Font("Arial", Font.PLAIN, 16));
    dayLabel.setBorder(new LineBorder(new Color(220, 220, 220)));

    // 样式逻辑
    if (dm != month) {
        // 非本月日期: 灰色
        dayLabel.setForeground(Color.LIGHT_GRAY);
        dayLabel.setBackground(new Color(250, 250, 250));
    } else {
        // 本月日期
        if (cal.get(Calendar.DAY_OF_WEEK) == Calendar.SUNDAY ||
            cal.get(Calendar.DAY_OF_WEEK) == Calendar.SATURDAY) {
            dayLabel.setForeground(new Color(200, 50, 50)); // 周末红色
        } else {
            dayLabel.setForeground(Color.BLACK);
        }
        dayLabel.setBackground(Color.WHITE);
    }
}
    
```

```

        // 如果是“今天”，特殊高亮
        if (dy == realToday.get(Calendar.YEAR) &&
            dm == realToday.get(Calendar.MONTH) &&
            dd == realToday.get(Calendar.DAY_OF_MONTH)) {
            dayLabel.setBackground(new Color(100, 149, 237));
            dayLabel.setForeground(Color.WHITE);
            dayLabel.setFont(new Font("Arial", Font.BOLD, 18));
            dayLabel.setBorder(new LineBorder(new Color(65, 105, 225), 2));
        }

        calendarPanel.add(dayLabel);

        // 日期加 1
        cal.add(Calendar.DAY_OF_MONTH, 1);
    }

    calendarPanel.revalidate();
    calendarPanel.repaint();
}

/**
 * 处理 Spinner 的数值变化
 */
private void updateFromSpinners() {
    int y = (int) yearSpinner.getValue();
    int m = (int) monthSpinner.getValue() - 1; // Spinner 是 1-12, Calendar
是 0-11

    // 只有当确实改变时才刷新，避免死循环
    if (y != currentCal.get(Calendar.YEAR) || m !=
currentCal.get(Calendar.MONTH)) {
        currentCal.set(Calendar.YEAR, y);
        currentCal.set(Calendar.MONTH, m);
        currentCal.set(Calendar.DAY_OF_MONTH, 1);
        refreshCalendar();
    }
}

@Override
public void actionPerformed(ActionEvent e) {
    String cmd = e.getActionCommand();

    if ("PREV".equals(cmd)) {
        currentCal.add(Calendar.MONTH, -1);
    }
}

```

```

    } else if ("NEXT".equals(cmd)) {
        currentCal.add(Calendar.MONTH, 1);
    } else if ("TODAY".equals(cmd)) {
        currentCal = Calendar.getInstance();
    }

    refreshCalendar();
}
}

```

在原有的万年历程序上进行修改，由于其作为万年历的功能已经足够完善，但是其 UI 界面还是过于简陋，因此参考我们完成计算器的步骤，我们首先对 UI 界面进行优化：

对于日期显示，我们原本使用直接以文本形式输出，这样的结果明显，不太美观，因此，我们换用 GridLayout 布局，让每一个日期都能以按钮的形式输出，同时，对于假期的日期，我们同时将其标红，同时将选中的日期进行高亮显示。

最后，我们添加回到今天按钮，一键按下，便可以一键回到今天，运行代码，修改后的代码如下：

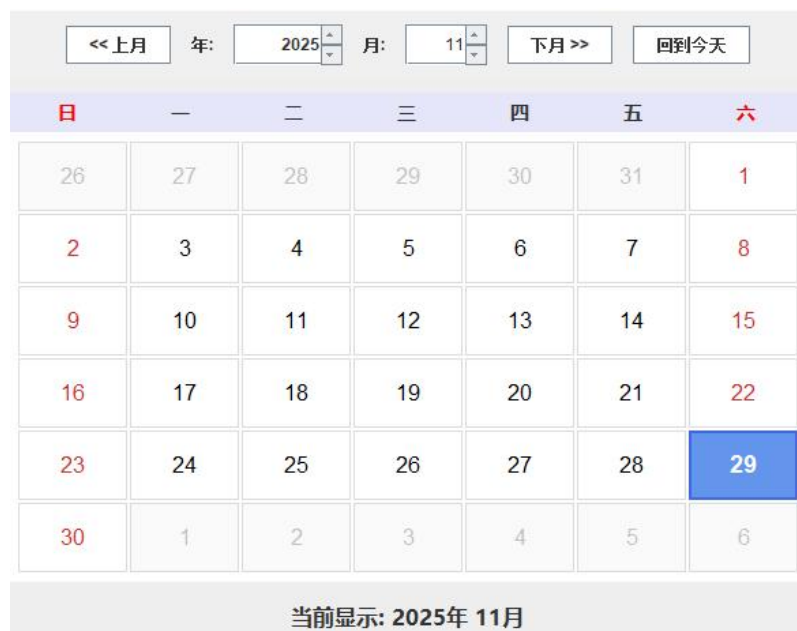


图 27 优化后的 UI 界面

我们将原本的

【实验心得】

1. 本次实验让我对 Java 的数组与字符串操作有了更扎实的掌握。通过随机数组的生成、最值与求和计算、排序，以及 arraycopy 和 copyof 方法的对比使用，我清晰理解了数组的初始化、遍历及拷贝机制；而 String 和 StringBuffer 类的实践，则让我熟悉了字符串长度、字符提取、子串截取等常用方法，尤其体会到 StringBuffer 在字符串修改（如插入、替换、追加）时的灵活性，也注意到单双引号在字符与字符串处理中的区别，这些细节对后续代码编写至关重要。

2. 类与对象的实验加深了我对面向对象编程核心思想的理解。从 Rectangle 类的封装实践中，我明确了 private 修饰符对属性保护的作用，以及构造器在初始化对象时的关键角色——特别是构造器不能用 void 修饰这一细节，通过报错与修正的过程印象尤为深刻。三角形、梯形和圆形类的设计，则让我学会了将业务逻辑（如三角形合法性判断、面积计算公式）封装到类的方法中，体会到封装带来的代码安全性和可维护性。

3. 方法重载与重写的练习让我掌握了 Java 中方法复用与扩展的核心机制。通过 plus 方法的重载实践，我直观理解了“同一类中方法名相同、参数列表不同”的重载规则，以及其简化相似功能方法调用的优势；而 Person 与 ZhangSan 类的继承案例，则清晰展示了方法重写中“子类覆盖父类方法”的特性，结合 super 和 this 的使用，进一步区分了继承关系中不同层级方法与变量的访问逻辑，对变量隐藏减少类耦合的作用有了实际感知。

4. 万年历的 GUI 实现让我体会到理论知识与实际应用的结合。通过 Swing 组件构建界面、绑定事件监听器、处理用户输入验证，我初步掌握了可视化程序的设计流程，也理解了 Calendar 类在日期计算中的核心作用（如获取月份天数、判断星期几）。将数组、字符串、类与对象等基础知识点整合到一个综合案例中，不仅巩固了所学，更让我认识到 Java 在实际应用开发中的灵活性和实用性。

课堂知识点总结：

1. 修饰词总结：

1) 用于数据类型与字面量

byte：字节型基本数据类型，占 1 字节，范围-128~127，用于存储小整数。

boolean：布尔型数据类型，仅 true 和 false 两个值，用于逻辑判断。

char：字符型基本数据类型，占 2 字节，存储单个 Unicode 字符（如'A'、'中'）。

double：双精度浮点型基本数据类型，用于存储高精度小数。

float：单精度浮点型基本数据类型，占 4 字节，声明时需加 f/F（如 3.14f）。

int：整型基本数据类型，占 4 字节，范围 $-2^{31} \sim 2^{31}-1$ ，存储整数。

long: 长整型基本数据类型, 占 8 字节, 声明时需加 l/L (如 1000000L)。

short: 短整型基本数据类型, 占 2 字节, 范围-32768~32767。

void: 表示方法无返回值, 如 `public void method() {}`。

true/false: 布尔类型的字面量, 分别表示 “真” 和 “假”。

null: 引用类型的 “空值”, 表示变量未指向任何对象。

2) 用于类、接口、继承与实现

class: 声明类, 是 Java 面向对象的基本单元 (包含属性和方法)。

interface: 声明接口, 是抽象类型, 可包含抽象方法、默认方法 (Java 8+) 等。

extends: 用于类的继承 (单继承) 或接口的继承 (多继承), 表示 “扩展”。

implements: 用于类实现接口, 需实现接口中所有抽象方法, 支持多实现。

abstract: 声明抽象类 (不能实例化) 或抽象方法 (无方法体, 需子类实现)。

3. 用于访问控制

private: 私有访问修饰符, 仅当前类内部可访问。

protected: 受保护访问修饰符, 当前类、同包类、子类可访问。

public: 公共访问修饰符, 任何地方均可访问, 权限最广。

package: 声明类所属的包, 用于组织类、避免命名冲突。

4. 用于流程控制

break: 跳出循环 (for/while/do-while) 或 switch 语句, 立即终止当前结构。

case: 用于 switch 语句, 定义分支条件, 匹配时执行对应代码。

continue: 在循环中跳过当前轮次剩余代码, 直接进入下一轮循环判断。

default: switch 的 “默认分支” (所有 case 不匹配时执行); 也可用于接口的默认方法 (Java 8+)。

do: 用于 do-while 循环, 先执行循环体, 再判断条件 (循环体至少执行一次)。

else: 与 if 搭配 (if-else/else if), 表示条件不成立时的分支。

for: 用于循环, 有普通 for、增强 for (foreach) 等形式, 重复执行代码块。

if: 条件判断, 条件为 true 时执行对应代码块。

switch: 多分支条件判断, 根据表达式值匹配 case 分支。

while: 先判断条件, true 则执行循环体 (循环体可能一次不执行)。

5. 用于异常处理

catch: 与 try/finally 搭配, 捕获 try 块中抛出的异常, 处理异常逻辑。

finally: try-catch 的 “最终块”, 无论是否异常, 其中代码必定执行 (常用于资源释放)。

try: 包裹可能抛异常的代码块, 启动异常处理流程。

throw: 手动抛出一个异常对象（如 `throw new RuntimeException("出错了")`）。

throws: 在方法签名后声明该方法可能抛出的异常，告知调用者需处理或继续声明。

6. 用于多线程与同步

synchronized: 实现线程同步，修饰方法或代码块，保证同一时间仅一个线程执行，解决线程安全问题。

volatile: 修饰变量，保证可见性（线程修改后其他线程立即感知）、禁止指令重排序，但不保证原子性。

7. 用于其他特殊场景

final: “最终的”，修饰类（不可继承）、方法（不可重写）、变量（变为常量，仅能赋值一次）。

import: 导入其他包的类 / 接口，避免写全类路径，简化代码。

instanceof: 判断对象是否为某类 / 子类的实例，返回布尔值（如 `obj instanceof String`）。

native: 声明“本地方法”，由其他语言（如 C/C++）实现，通过 JNI 调用。

new: 创建对象，在堆内存分配空间并调用构造器初始化。

return: 从方法返回值（有返回类型的方法必须返回对应值）；也可提前退出无返回值的方法。

static: “静态的”，修饰变量（类变量，所有对象共享）、方法（类方法，可直接通过类名调用）、代码块（类加载时执行）。

super: 引用父类成员（调用父类构造器 `super()`、访问父类被隐藏的属性 / 方法）。

this: 引用“当前对象”，区分成员变量与局部变量、调用本类其他构造器（`this()`）等。

transient: 修饰对象属性，序列化时（如实现 `Serializable`）该属性不会被保存（不参与序列化）。